Exchange Specification

This document describes the simulated exchange model for Challenge 02.

Overview

The exchange operates a **central limit order book (CLOB)** for 200 symbols. All operations are **single-threaded** – there is no concurrency. Your `MultiOrderBook` receives a sequential stream of events and must maintain consistent state after each one.

Symbols

- 200 symbols numbered `0` to `199`
- 50 "hot" symbols (`0-49`) generate ~90% of message traffic
- The remaining 150 "cold" symbols generate ~10%
- Prices cluster near each symbol's mid-price (80% within 100 ticks)

Order Types

All orders are **limit orders** with a price and quantity. There are no market orders, stop orders, or iceberg orders.

Sides

- `0` = **bid** (buy)
- `1` = **ask** (sell)

Our Orders vs Exchange Orders

Your `MultiOrderBook` must track two categories of orders:

Exchange feed orders arrive via `add_order()`, `modify_order()`, and `cancel_order()`. These include everyone's orders on the exchange – both from other participants and from us (once the exchange confirms them).

Our orders are initiated by your strategy via `send_order()`, `modify_our_order()`, and `cancel_our_order()`. These go through the `Venue` interface and appear in the feed after a short delay.

Venue Round-Trip

When you call `venue.send_order(our_id, symbol, side, price, qty)`:

1. The venue returns an `exchange_id` immediately
2. Your order does **NOT** enter the book at this point
3. After a short delay (3-10 operations), the exchange feed delivers `add_order(exchange_id, symbol, side, price, qty)`
4. Only then should the order appear in your book

You must remember the mapping from `our_id` to `exchange_id` so you can recognize your order when it arrives in the feed.

Modify Semantics

When you call `venue.modify_order(exchange_id, new_price, new_qty)`:

Quantity decrease (same price)

- Returns the **same** `exchange_id`
- Order **keeps** its queue position
- Feed delivers: `modify_order(exchange_id, new_qty)`

Price change or quantity increase

- Returns a **new** `exchange_id`
- Order **loses** its queue position
- Feed delivers: `cancel_order(old_exchange_id)` followed by `add_order(new_exchange_id, symbol, side, new_price, new_qty)`
- The cancel and add arrive as consecutive feed events

Queue Position (FIFO)

Orders at the same price level are filled in FIFO order. The queue position of an order is defined as:

- **index**: number of orders ahead of it at the same price level (0 = front)
- **qty_ahead**: total quantity of all orders ahead of it

`get_queue_position(our_id)` returns `{-1, 0}` if: - The `our_id` is unknown - The order has been sent but not yet confirmed (not in the book)

Level Queries

`best_bid(symbol)` / `best_ask(symbol)`

Returns a `TopLevel` struct with: - `price`: the best price (highest bid, lowest ask), or `0` if empty - `qty`: total quantity at that price level - `count`: number of orders at that price level

`get_top_levels(symbol, side, n, out)`

Writes up to `n` `TopLevel` entries into `out[]`, ordered from best to worst: - Bids: highest price first - Asks: lowest price first - Returns the number of levels actually written (may be less than `n`)

`volume_near_best(symbol, side, depth)`

Returns total quantity within `depth` ticks of the best price (inclusive). For bids: prices in `[best - depth + 1, best]`. For asks: prices in `[best, best + depth - 1]`. Returns `0` if no orders on that side.

Workload Characteristics

The benchmark prefills the book with **500,000 orders** across all 200 symbols before timing begins. During the timed phase, the operation mix is approximately:

Operation	Percentage
add_order (feed)	35%
cancel_order (feed)	20%
modify_order (feed, qty-down)	10%
send_order (ours)	5%
modify_our_order	3%
cancel_our_order	2%
best_bid / best_ask	10%
get_top_levels	5%
volume_near_best	5%

Operation	Percentage
get_queue_position	5%

Scoring

The primary metric is **P99 latency** in TSC cycles per operation. Every operation (add, cancel, modify, query) is individually timed. Solutions with allocation spikes, hash table resizes, or tree rebalancing during the timed phase will be penalized even if their average latency is fast.

Constraints

- Single-threaded execution – no threads, no async
- Libraries: anything on the pre-installed list at https://hftuniversity.com/challenges/libraries. Boost headers and Abseil are linked automatically; for the others add a `solution/libraries.cmake` as described there
- Time limit: 120 seconds
- Memory limit: 2048 MB